

CPS 150 Review Session #3

Branching

Review Activity

What would be the result of the following String methods?

```
String greeting = "Hello, World";
```

```
greeting.toUpperCase();  
greeting.length();  
greeting.charAt(4);  
greeting.substring(2, 8);  
greeting.substring(7);
```

What is Branching?

Branching allows us to have conditional flow within our programs (i.e. `if X is true,`
`do Y, but if X is not true, do Z`)

Booleans

Booleans store `true` or `false` value

Think of it like a light switch (either on or off, no in between)

They are the building block of branching, allowing us to check a condition (i.e. `x` in `if x is true, ...`)

Comparison Logic

Allow us to compare two values, and return `true` or `false`

Java	Math Equivalent
<code>==</code>	$=$
<code><</code>	$<$
<code>></code>	$>$
<code>!=</code>	$\neq$
<code><=</code>	$\leq$
<code>>=</code>	$\geq$

Logical Operators

Allow us to chain together several Boolean expressions

Java	English Equivalent	Math Equivalent
<code>&&</code>	and	$\wedge$
<code> </code>	or	$\vee$
<code>!</code>	not	$\neg$

Logic and Strings

Since Strings are objects, we need to compare value slightly differently.

```
String greeting = "Hello, World";  
  
greeting.isEmpty();  
greeting.equals("hello, world");  
greeting.equalsIgnoreCase("hello, world");
```

Evaluating Expressions

```
String myString = "Hello";  
boolean myBool = false;  
  
// Expression 1  
(15 < 27 && (23 != 15 + 8))  
  
// Expression 2  
(34 < 0 || myBool)  
  
// Expression 3  
(!myString.equals("Goodbye"))  
  
// Expression 4  
(myString.equalsIgnoreCase("HELLO") && (2 > 4 || !myBool))
```

if Block

Will execute if only the condition is true

```
if (condition) {  
    // execute this code if condition is true  
}
```

else Block

Fallback if the `if` statement is false

```
if (condition) {  
    // execute this code if condition is true  
} else {  
    // execute this code if condition is false  
}
```

else if Block

Allows us to chain together multiple condition checks

```
if (condition1) {  
    // execute this code if condition1 is true  
} else if (condition2) {  
    // execute this code if condition1 is false and condition2 is true  
}
```

Putting It All Together

```
if (condition1) {  
    // execute this code if condition1 is true  
} else if (condition2) {  
    // execute this code if condition1 is false and condition2 is true  
}  
// additional else if expressions if desired  
else {  
    // execute this code if condition1, condition2, ... are false  
}
```

Nesting Conditional Statements

We can put additional conditional statements within an existing conditional statement

```
if (condition1) {  
    if (condition2) {  
        // execute if condition1 and condition2 are true  
    } else if (condition3) {  
        // execute if condition1 and condition3 are true  
    } else {  
        // execute if condition1 is true and condition2 and condition 3 are false  
    }  
} // include additional logic structures
```

switch Structure

Switch allows us to quickly compare values and do a lot of value comparisons in a relatively simpler syntax

```
switch(expression) {  
    case x:  
        // execute if expression == x  
        break;  
    case y:  
        // execute if expression == y  
        break;  
    // additional cases here  
    default:  
        // execute if no value match is found (else equivalent)  
        break;  
}
```

Practice Problem

Write a program that determines if a user-input year is a leap year.

Leap Year Rules:

1. A year is a leap year if it is divisible by 4
2. However, if the year is divisible by 100, it is not a leap year, unless it is also divisible by 400